Tarea 2: Torneo de Komikan Concurrente

Informe de diseño, Sistemas Operativos
Valentín Aravena y Gustavo Femenias

1. Cómo protegimos el tablero

Todo el programa corre en un solo proceso y cada cosa que actúa por su cuenta es un hilo: cada jugador que llega, el servidor que arma las parejas, el juez de cada mesa, y las 13 fichas de cada mesa (el puma y los 12 perros). Usamos hilos y no procesos porque así el tablero ya queda compartido sin hacer nada especial, ya que todos los hilos de un programa ven las mismas variables.

El problema es que las 13 fichas de una mesa tocan el mismo tablero al mismo tiempo. Si dos perros saltan a la misma casilla vacía a la vez, los dos podrían creer que la ganaron y el tablero quedaría mal. Para que eso no pase, cada mesa tiene un mutex para todo el tablero, y nadie puede mirar ni cambiar el tablero sin tenerlo.

Cada ficha hace siempre lo mismo, en tres pasos:

1. Toma el mutex, mira el tablero, elige a dónde quiere ir y suelta el mutex.

2. Piensa entre 50 y 300 milisegundos, sin el mutex, para no hacer esperar a las demás.

3. Vuelve a tomar el mutex e intenta moverse.

En el tercer paso el juez revisa el movimiento con el tablero como está en ese momento, porque mientras la ficha pensaba otra pudo haberse movido. Si la casilla sigue libre, la ficha se mueve. Si otra ficha llegó antes, el movimiento se rechaza y la ficha vuelve a intentar en la siguiente vuelta. Como revisar y mover pasa entero con el mutex tomado, es imposible que dos fichas se queden con la misma casilla: la primera que toma el mutex se la queda y la otra la encuentra ocupada.

Lo demás que se comparte lo resolvimos así:

1. Máximo M mesas a la vez: un semáforo que parte en M. Cada partida nueva le resta uno y cada partida que termina le suma uno. Si llega a cero, el servidor se queda dormido esperando una mesa, sin gastar CPU.

2. La sala de espera: un mutex y una variable de condición. Cada jugador que llega se anota y despierta al servidor, que duerme hasta que haya dos.

3. El juez: espera dormido en una variable de condición hasta que una ficha le avise que alguien ganó o que un jugador se cayó.

4. Un jugador que se cae: lo simulamos haciendo que su ficha termine teniendo el mutex tomado, que es lo peor que puede pasar. Con un mutex normal, el tablero quedaría trancado para siempre. Por eso usamos un mutex "robusto": cuando el siguiente lo pide, en vez de quedarse esperando, el sistema le avisa que el dueño anterior murió. Esa ficha deja el mutex usable de nuevo y avisa al juez, que le da la victoria al otro jugador por abandono.

5. Ctrl+C: solo un hilo (el portero) recibe la señal. Cuando llega, avisa a todos que hay que cerrar. El servidor deja de armar parejas, los jueces cancelan sus partidas y al final se destruyen todos los mutex, semáforos y variables de condición, y se libera la memoria.

2. Qué podría salir mal y cómo lo evitamos

Deadlock (dos hilos esperándose para siempre). Pasa cuando un hilo tiene un mutex y espera otro que tiene un segundo hilo, y ese segundo hilo espera el primero. Dentro de una mesa no puede pasar, porque hay un solo mutex y nadie pide otro mientras lo tiene. Entre mutex distintos (el de una mesa, el del servidor y el de cerrar) tampoco, porque nunca se tienen a la vez el de una mesa y el del servidor, y con el de cerrar tomado no se pide ningún otro. El otro caso de deadlock sería el jugador que se cae con el mutex tomado, y eso lo resuelve el mutex robusto que explicamos arriba. Por último,

para que el juez no se quede dormido si se pierde un aviso, se despierta solo cada 200 milisegundos a revisar.

Livelock (hilos que se mueven pero nadie avanza). Cuando dos fichas quieren la misma casilla, una gana y la otra es rechazada, así que siempre alguien avanza. Además, como cada ficha piensa un tiempo al azar, no vuelven a chocar siempre al mismo ritmo. Lo que sí podía pasar es que una partida no terminara nunca, con las fichas dando vueltas sin que nadie gane. Para eso pusimos un límite de movimientos: si se llega a 500, la partida es empate.

Starvation (un hilo que nunca le toca). El mutex del tablero no atiende por orden de llegada, así que en teoría una ficha podría perder siempre. En la práctica el mutex se ocupa unos pocos microsegundos y cada ficha lo pide cada 50 a 300 milisegundos, así que casi nunca hay que esperarlo. Si una ficha es rechazada, lo vuelve a intentar en la siguiente vuelta. Las parejas que esperan mesa tampoco se quedan atrás para siempre: la sala de espera es una fila y las mesas se van dando en el orden en que llegaron los jugadores.

3. Por qué lo hicimos así

Complejidad. Un solo mutex por tablero es fácil de entender y de explicar. La otra opción era un mutex por casilla, para que fichas lejanas se pudieran mover al mismo tiempo. Pero cuando el puma come toca tres casillas, y habría que tomar tres mutex siempre en el mismo orden para no caer en un deadlock. Además, para saber si el puma está encerrado habría que tomar los mutex de todas sus casillas vecinas. Era mucho más complicado y más fácil equivocarse, y la ganancia no se notaría.

Rendimiento. Lo que más demora es el tiempo que cada ficha piensa (50 a 300 milisegundos), no el mutex, que se ocupa unos microsegundos. Una partida normal tiene unos 140 movimientos y dura unos 4 segundos. En la prueba de estrés, con 10.000 jugadores, 200 mesas y fichas que piensan entre 1 y 10 milisegundos, se jugaron 5.000 partidas y más de un millón de movimientos en unos 20 segundos, y nunca hubo más de M mesas a la vez. También vimos que los que esperan están dormidos de verdad: con 6 jugadores y 1 mesa el programa duró 15 segundos pero usó menos de una décima de segundo de CPU. Al revisarlo con valgrind no quedaron fugas de memoria ni archivos abiertos, incluso con jugadores que se caen y con Ctrl+C.

Qué tanto se puede hacer al mismo tiempo. Las mesas juegan totalmente en paralelo, porque cada una tiene su propio mutex. Dentro de una mesa, las 13 fichas piensan al mismo tiempo y lo único que se hace de a una es aplicar cada movimiento. El costo de dejar que piensen sin el mutex es que entre un cuarto y un tercio de los intentos se rechazan porque otra ficha llegó antes. La otra opción era que pensaran con el mutex tomado, y así no habría rechazos, pero entonces cada ficha que piensa dejaría esperando a las otras 12 y el juego terminaría siendo por turnos, que es justo lo que el torneo no quiere.